



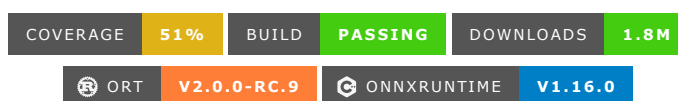
ort v1.16.3

A safe Rust wrapper for ONNX Runtime 1.20 - Optimize and accelerate machine learning inference & training

#ai #machine-learning #ml

[Readme](#)[39 Versions](#)[Dependencies](#)[Dependents](#)

Rust bindings for ONNX Runtime



ort is an (unofficial) [ONNX Runtime](#) 1.16 wrapper for Rust based on the now inactive [onnxruntime-rs](#). ONNX Runtime accelerates ML inference on both CPU & GPU.

See [the docs](#) for more detailed information and the [examples](#). If you have any questions, feel free to ask in the [#ort-discussions](#) and related channels in the [pyke Discord server](#) or in [GitHub Discussions](#).

- [Feature comparison](#)
- [Cargo features](#)
- [How to get binaries](#)
 - [Strategies](#)
- [Execution providers](#)
- [Projects using ort](#) ❤️
- [FAQ](#)
 - I'm using a non-CPU execution provider, but it's still using the CPU!
 - My app exits with "status code 0xc000007b" without logging anything!
 - "thread 'main' panicked at 'assertion failed: (left != right)'"
- [Shared library hell](#)

Feature comparison

Feature comparison	ort	ors	onnxruntime-rs
Upstream version	v1.16.0	v1.12.0	v1.8
dlopen()?	✓	✓	✗
Execution providers?	✓	✗	✗
IOBinding?	✓	✗	✗
String tensors?	✓	✗	⚠ input only
Multiple output types?	✓	✓	✗
Multiple input types?	✓	✓	✗
In-memory session?	✓	✓	✓

Feature comparison	 ort	 ors	 onnxruntime-rs
WebAssembly?	✓	✗	✗

Cargo features

Note: For developers using `ort` in a **library** (if you are developing an *app*, you can skip this part), it is heavily recommended to use `default-features = false` to avoid bringing in unnecessary bloat. Cargo features are **additive**. Users of a library that requires `ort` with default features enabled **will not be able to remove those features**, and if the library isn't using them, it's just adding unnecessary bloat and inflating compile times. Instead, you should enable `ort`'s default features in your *dev dependencies only*. Disabling default features will disable `download-binaries`, so you should instruct downstream users to include `ort = { version = "...", features = [ "download-binaries" ] }` in their dependencies if they need it.

- **download-binaries (default):** Enables downloading binaries via the `download` [strategy](#). If disabled, the default behavior will be the `system` [strategy](#).
- **copy-dylibs (default):** Copies the dynamic libraries to the Cargo build folder - see [shared library hell](#).
- **half (default):** Enables support for using `float16` / `bfloat16` tensors in Rust.
- **fetch-models** : Enables fetching models from the ONNX Model Zoo. Useful for quick testing with some common models like YOLOv4, GPT-2, and ResNet. Not recommended in production.
- **load-dynamic** : Loads the ONNX Runtime binaries at runtime via `dlopen()` without a link dependency on them. The path to the binary can be controlled with the environment variable `ORT_DYLIB_PATH=/path/to/libonnxruntime.so`. This is heavily recommended, as it mitigates the [shared library hell](#).

How to get binaries

You can use either the 'traditional' way, involving a [strategy](#), or the new (and preferred) way, using `load-dynamic`.

- **Strategies:** Links to provided or downloaded dynamic libraries; see below. This is useful for static linking and quick prototyping (making use of the `download` [strategy](#)), but might cause more headaches than `load-dynamic`.
- **load-dynamic** : This *doesn't link to any dynamic libraries*, instead loading the libraries at runtime using `dlopen()`. This can be used to control the path to the ONNX Runtime binaries (meaning they don't always have to be directly next to your executable), and avoiding the [shared library hell](#). To use this, enable the `load-dynamic` Cargo feature, and set the `ORT_DYLIB_PATH` environment variable to the path to your `onnxruntime.dll` / `libonnxruntime.so` / `libonnxruntime.dylib` - you can also use relative paths like `ORT_DYLIB_PATH=./libonnxruntime.so` (it will be relative to the executable). For convenience, you should download or compile ONNX Runtime binaries, put them in a permanent location, and set the environment variable permanently.

Strategies

There are 2 'strategies' for obtaining and linking ONNX Runtime binaries. The strategy can be set with the `ORT_STRATEGY` environment variable.

- **download (default):** Downloads prebuilt ONNX Runtime from Microsoft. Only a few execution providers are available for download at the moment, namely CUDA and TensorRT. These binaries [may collect telemetry](#). In the future, pyke may provide binaries with telemetry disabled and more execution providers available.
- **system** : Links to ONNX Runtime binaries provided by the system or a path pointed to by the `ORT_LIB_LOCATION` environment variable. `ort` will automatically link to static or dynamic libraries depending on what is available in the `ORT_LIB_LOCATION` folder.

Execution providers

To use other execution providers, you must explicitly enable them via their Cargo features, listed below. **Some EPs are not currently implemented due to a lack of hardware for testing; please open an issue if your desired EP has a ⚠️**

-  **cuda** : Enables the CUDA execution provider for Maxwell (7xx) NVIDIA GPUs and above. Requires CUDA v11.6+.
-  **tensorrt** : Enables the TensorRT execution provider for GeForce 9xx series NVIDIA GPUs and above; requires CUDA v11.4+ and TensorRT v8.4+.
-  **openvino** : Enables the OpenVINO execution provider for 6th+ generation Intel Core CPUs.
-  **onednn** : Enables the Intel oneDNN execution provider for x86/x64 targets.
-  **directml** : Enables the DirectML execution provider for Windows x86/x64 targets with dedicated GPUs supporting DirectX 12.
-  **qnn** : Enables the Qualcomm AI Engine Direct SDK execution provider for Qualcomm chipsets.
-  **nnapi** : Enables the Android Neural Networks API (NNAPI) execution provider. (needs testing - [#45](#))
-  **coreml** : Enables the CoreML execution provider for macOS/iOS targets.
-  **xnnpack** : Enables the **XNNPACK** backend for WebAssembly and Android.
-  **migraphx** : Enables the MIGraphX execution provider AMD GPUs.
-  **rocm** : Enables the ROCm execution provider for AMD ROCm-enabled GPUs. ([#16](#))
-  **acl** : Enables the ARM Compute Library execution provider for multi-core ARM v8 processors.
-  **axmnn** : Enables the ArmNN execution provider for ARM v8 targets.
-  **tvm** : Enables the **preview** Apache TVM execution provider.
-  **rknpu** : Enables the RKNPU execution provider for Rockchip NPUs.
-  **vitis** : Enables Xilinx's Vitis-AI execution provider for U200/U250 accelerators.
-  **cann** : Enables the Huawei Compute Architecture for Neural Networks (CANN) execution provider.

Note that the `download` strategy only provides some execution providers, namely CUDA and TensorRT for Windows & Linux. You'll need to compile ONNX Runtime from source and use the `system` strategy to point to the compiled binaries to enable other execution providers.

Execution providers will attempt to be registered in the order they are passed, silently falling back to the CPU provider if none of the requested providers are available. If you must know whether an EP is available, you can use `ExecutionProvider::cuda().is_available()`.

For prebuilt Microsoft binaries, you can enable the CUDA or TensorRT execution providers for Windows and Linux via the `cuda` and `tensorrt` Cargo features respectively. Microsoft does not provide prebuilt binaries for other execution providers, and thus enabling other EP features will fail when `ORT_STRATEGY=download`. To use other execution providers, you must build ONNX Runtime from source.

Projects using ort

[open a PR](#) to add your project here 🌟

- **Twitter** uses `ort` to serve homepage recommendations to hundreds of millions of users.
- **Bloop** uses `ort` to power their semantic code search feature.
- **pyke Diffusers** uses `ort` for efficient Stable Diffusion image generation on both CPUs & GPUs.
- **edge-transformers** uses `ort` for accelerated transformer model inference at the edge.
- **Ortex** uses `ort` for safe ONNX Runtime bindings in Elixir.

FAQ

I'm using a non-CPU execution provider, but it's still using the CPU!

`ort` is designed to fail gracefully when an execution provider is not available. It logs failure events through `tracing`, thus you'll need a library that subscribes to `tracing` events to see the logs. The simplest way to do this is to use `tracing-subscriber`.

Add `tracing-subscriber` to your Cargo.toml:

[dependencies]

```
tracing-subscriber = { version = "0.3", features = [ "env-filter", "fmt" ] }
```

In your main function:

```
fn main() {
    tracing_subscriber::fmt::init();
}
```

Set the environment variable `RUST_LOG` to `ort=debug` to see all debug messages from `ort`; this will look like:

- Windows (PowerShell): `$env:RUST_LOG = 'ort=debug'; cargo run`
- Windows (Command Prompt): use PowerShell;)
- macOS & Linux: `RUST_LOG="ort=debug" cargo run`

My app exits with "status code 0xc000007b" without logging anything!

You probably need to copy the ONNX Runtime DLLs to the same path as the executable.

- If you are running a binary (`cargo run`), copy them to e.g. `target/debug`
- If you are running an example (`cargo run --example xyz`), copy them to e.g. `target/debug/examples`
- If you are running tests (`cargo test`), copy them to e.g. `target/debug/deps`

Alternatively, you can use the [load-dynamic feature](#) to avoid this.

"thread 'main' panicked at 'assertion failed: (left != right)'"

Most of the time this is because Windows ships its own (typically older) version of ONNX Runtime. Make sure you've copied the ONNX Runtime DLLs to the same folder as the exe.

Shared library hell

If using shared libraries (as is the default with `ORT_STRATEGY=download`), you may need to make some changes to avoid issues with library paths and load orders, or preferably use the [load-dynamic feature](#) to avoid all of this.

Windows

Some versions of Windows come bundled with an older version of `onnxruntime.dll` in the System32 folder, which will cause an assertion error at runtime:

```
The given version [14] is not supported, only version 1 to 13 is supported in this build.
thread 'main' panicked at 'assertion failed: `(left != right)`
  left: `0x0`,
 right: `0x0``, src\lib.rs:114:5
note: run with `RUST_BACKTRACE=1` environment variable to display a backtrace
```

The fix is to copy the ONNX Runtime DLLs into the same directory as the binary, since [DLLs in the same folder as the main executable resolves before system DLLs](#). `ort` can automatically copy the DLLs to the Cargo target folder with the `copy-dylibs` feature, though this fix only works for *binary* Cargo targets (`cargo run`). When running tests/benchmarks/examples for the first time, you'll have to manually copy the `target/debug/onnxruntime*.dll` files to `target/debug/deps/` for tests & benchmarks or `target/debug/examples/` for examples.

Linux

Running a binary via `cargo run` should work without `copy-dylibs`. If you'd like to use the produced binaries outside of Cargo, you'll either have to copy `libonnxruntime.so` to a known lib location (e.g. `/usr/lib`) or enable `rpath` to load libraries from the same folder as the binary and place `libonnxruntime.so` alongside your binary.

In `Cargo.toml`:

```
[profile.dev]
rpath = true
```

```
[profile.release]
```

```
    rpath = true
```

```
    # do this for all profiles
```

In `.cargo/config.toml`:

```
[target.x86_64-unknown-linux-gnu]
```

```
rustflags = [ "-Clink-args=-Wl,-rpath,\\$ORIGIN" ]
```

```
    # do this for all Linux targets as well
```

macOS

macOS has the same limitations as Linux. If enabling `rpath`, note that the `rpath` should point to `@loader_path` rather than `$ORIGIN`:

```
# .cargo/config.toml
```

```
[target.x86_64-apple-darwin]
```

```
rustflags = [ "-Clink-args=-Wl,-rpath,@loader_path" ]
```

Metadata

 over 1 year ago

 v1.70.0

 MIT OR Apache-2.0

 698 KiB

Install

Run the following Cargo command in your project directory:

```
cargo add ort
```

Or add the following line to your `Cargo.toml`:

```
ort = "1.16.3"
```

Homepage

 ort.pyke.io

Documentation

 docs.rs/ort/1.16.3

Repository

 github.com/pykeio/ort

Owners

 [decahedron1](#)

Categories

- [Algorithms](#)
- [Mathematics](#)
- [Science](#)

Stats Overview

↓ **1,817,544**

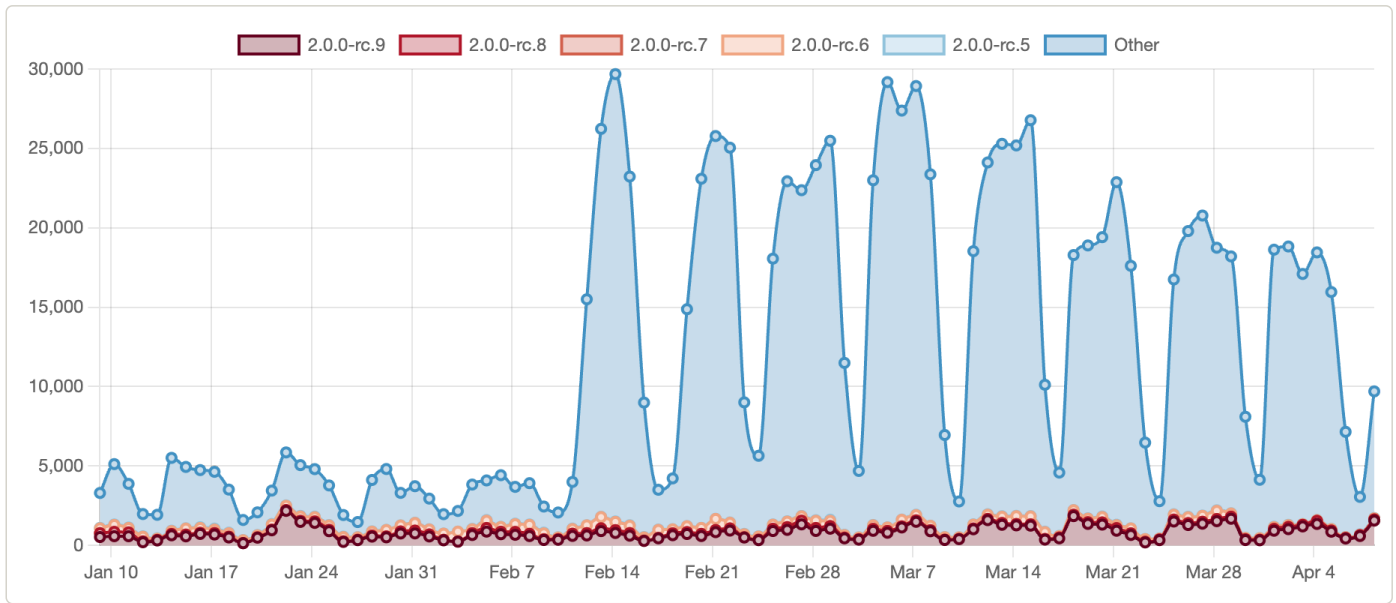
Downloads all time

📦 **39**

Versions published

Downloads over the last 90 days

Display as Stacked ▾



Rust

rust-lang.org

[Rust Foundation](#)

[The crates.io team](#)

Get Help

[The Cargo Book](#)

[Support](#)

[System Status](#)

[Report a bug](#)

Policies

[Usage Policy](#)

[Security](#)

[Privacy Policy](#)

[Code of Conduct](#)

[Data Access](#)

Social

[rust-lang/crates.io](#)

[#t-crates-io](#)

[@cratesiostatus](#)